

# รายงานผลการทดลอง Lab 9: Spring Boot - Transaction

วิชา การพัฒนาเว็บเซอร์วิสและเว็บแอปพลิเคชัน

|               |                  |
|---------------|------------------|
| ชื่อ-นามสกุล: | แทนคุณ พันธุ์กุล |
| รหัสนักศึกษา: | 673380301-0      |
| Section:      | 1                |

## 1. การสร้าง Account (POST /accounts)

The screenshot displays a REST client interface for a POST request to `http://localhost:8080/accounts`. The request body is a JSON object:

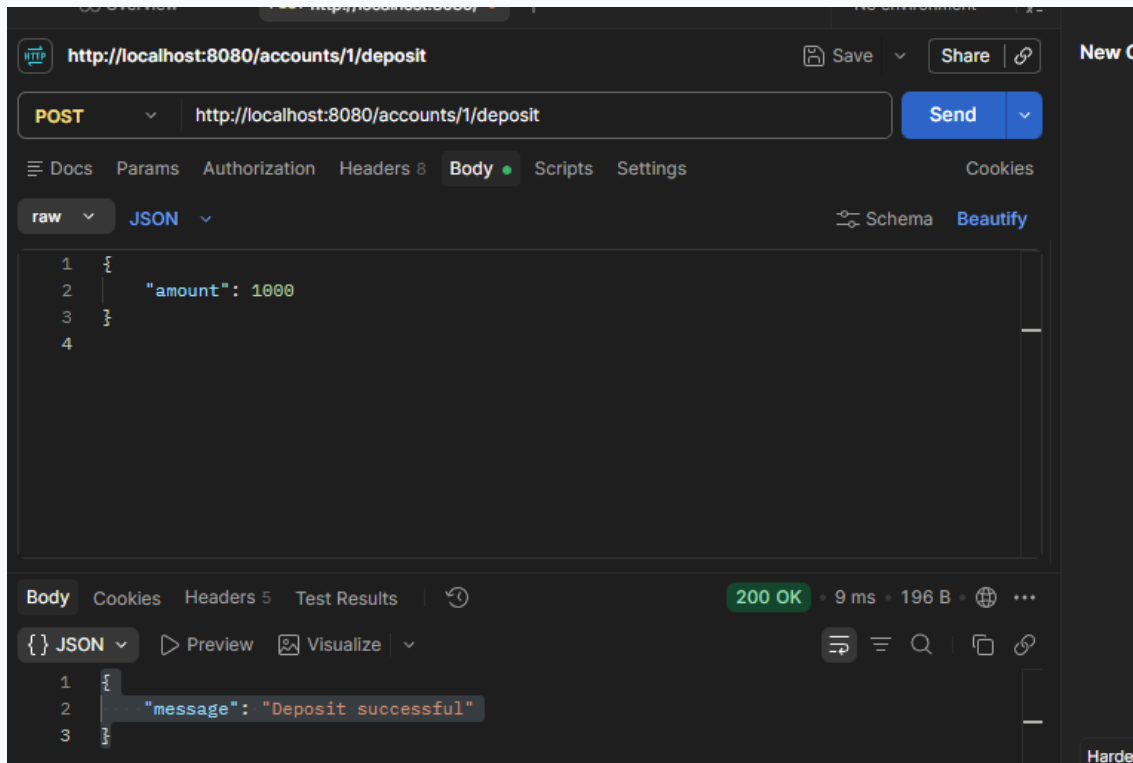
```
{  "accountNumber": "1234567890",  "ownerName": "Tank",  "balance": 0}
```

The response is a `200 OK` status with a response time of `16 ms` and a body size of `234 B`. The response body is a JSON object:

```
{  "accountNumber": "1234567890",  "ownerName": "Tank",  "balance": 0.0,  "id": 2}
```

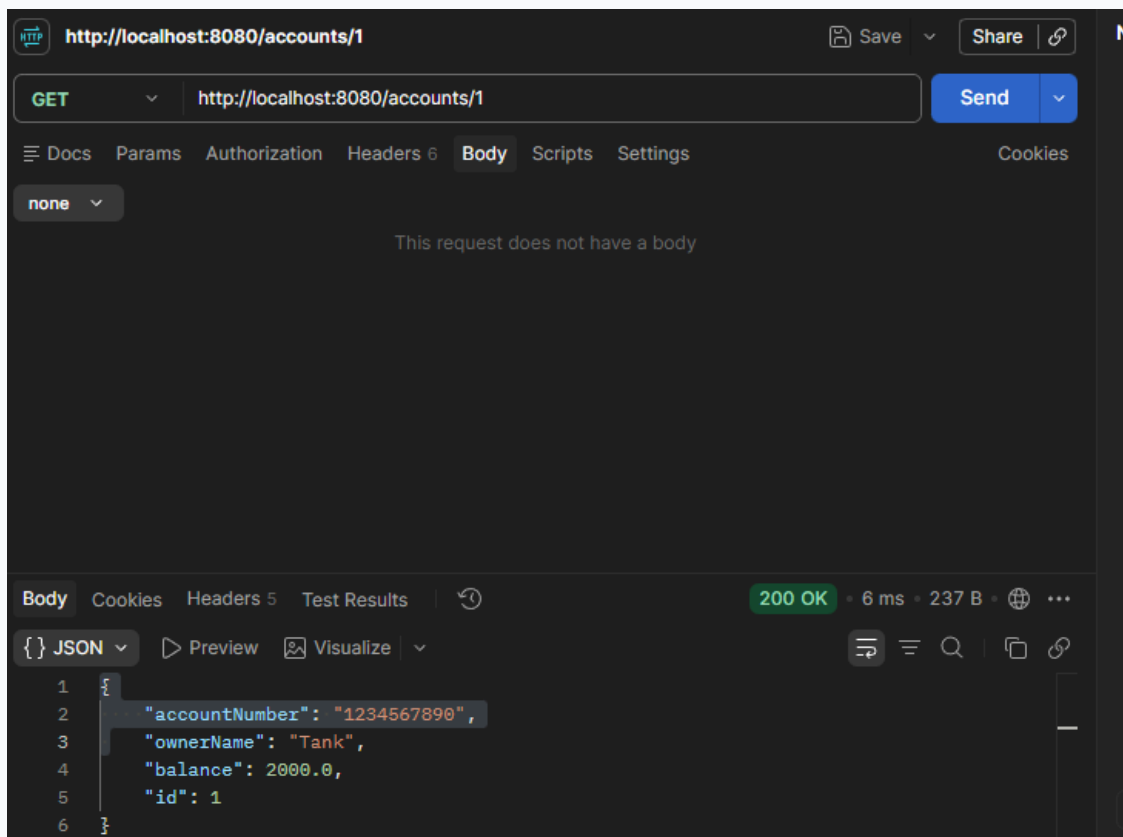
**อธิบายผลการทดลอง:** ทำการส่งคำขอแบบ POST ไปที่ Endpoint `http://localhost:8080/accounts` พร้อมส่งข้อมูล JSON ได้แก่ `accountNumber`, `ownerName` และ `balance` เริ่มต้นที่ 0 บาท ผลลัพธ์พบว่าเซิร์ฟเวอร์ตอบกลับสถานะ 200 OK โดยระบบทำการบันทึกข้อมูลลงในตาราง `account` ของฐานข้อมูล PostgreSQL และสร้างรหัส `id: 1` ให้กับบัญชีใหม่อย่างถูกต้อง

## 2. การฝากเงินสำเร็จ (POST /accounts/{id}/deposit)



**อธิบายผลการทดลอง:** ทำการส่งคำขอแบบ POST ไปที่ `http://localhost:8080/accounts/1/deposit` พร้อมส่ง Body เป็น JSON ระบุยอดเงินที่ต้องการฝาก `amount: 1000` บาท ระบบประมวลผลผ่าน `DepositService.deposit()` เพื่อทำการอัปเดตยอดเงินในบัญชีและบันทึกข้อมูลธุรกรรมการฝาก ผลลัพธ์ได้ Status 200 OK พร้อมข้อความตอบกลับ `Deposit successful`

## 3. ตรวจสอบ Account (GET /accounts/{id})



**อธิบายผลการทดลอง:** ทำการส่งคำขอแบบ GET ไปที่ `http://localhost:8080/accounts/1` เพื่อตรวจสอบข้อมูลล่าสุดของบัญชี ผลลัพธ์แสดงข้อมูลบัญชีพบว่ายอดเงินคงเหลือ (balance) เปลี่ยนแปลงจากเดิม 1000.0 บาท เพิ่มขึ้นเป็น 2000.0 บาท ถูกต้องตรงตามจำนวนเงินที่ได้ทำรายการฝาก

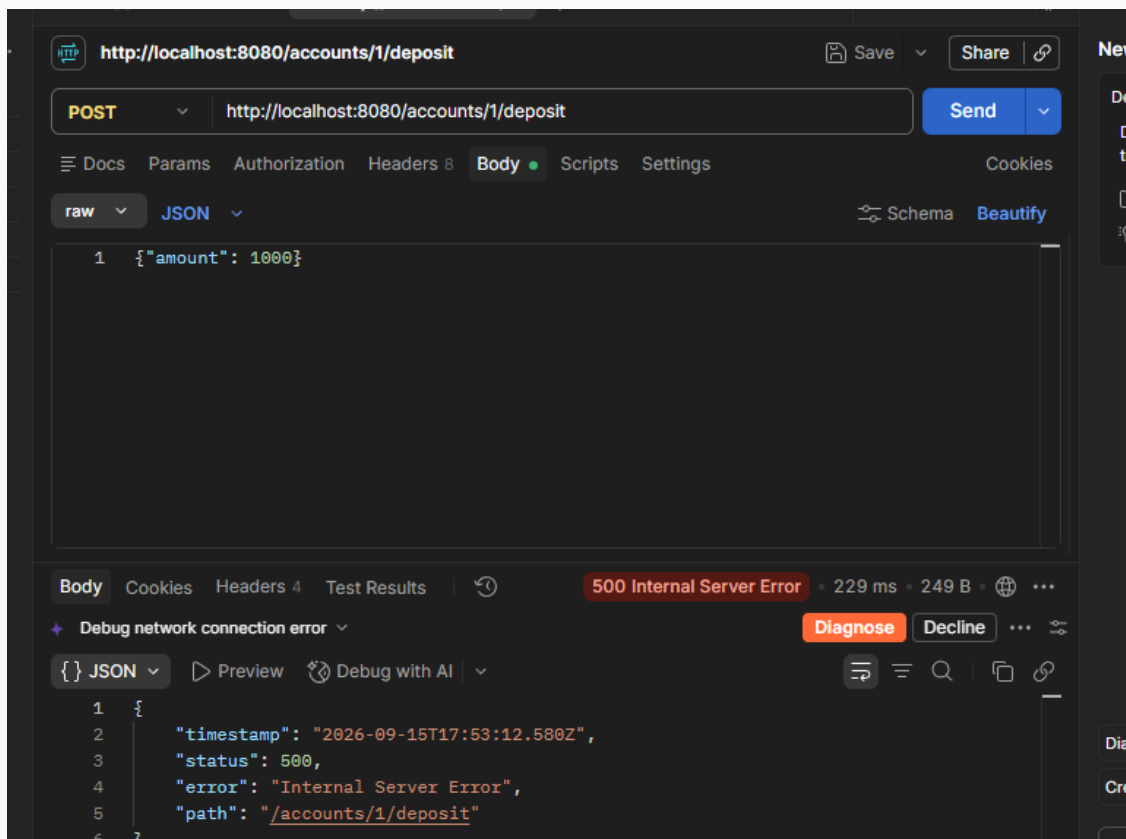
#### 4. ตรวจสอบ DepositTransaction ในฐานข้อมูล

The screenshot shows the pgAdmin interface. The 'Query' tab is active, displaying the SQL query: `SELECT * FROM deposit_transaction;`. The 'Data Output' tab is also visible, showing the results of the query in a table format. The table has four columns: 'id' (PK) bigint, 'amount' double precision, and 'account\_id' bigint. There are two rows of data: (1, 1000, 1) and (2, 1000, 1).

|   | id<br>[PK] bigint | amount<br>double precision | account_id<br>bigint |
|---|-------------------|----------------------------|----------------------|
| 1 | 1                 | 1000                       | 1                    |
| 2 | 2                 | 1000                       | 1                    |

**อธิบายผลการทดลอง:** ทำการเปิด pgAdmin เพื่อ Query ดูข้อมูลในตาราง deposit\_transaction ผ่านคำสั่ง `SELECT * FROM deposit_transaction;` พบรายการข้อมูลการฝากเงินจำนวน 1 แถว โดยมีค่า id: 1, amount: 1000 และ account\_id: 1 ซึ่งแสดงว่าระบบมีการบันทึกประวัติการฝากเงินและผูก Foreign Key ไปยังตาราง account อย่างสมบูรณ์

## 5. ทดลอง Rollback (จำลอง Error ด้วย throw new RuntimeException)



**อธิบายผลการทดลอง:** ทำการเพิ่มคำสั่ง `throw new RuntimeException("Test Rollback");` ไว้ที่บรรทัดสุดท้ายของเมธอด `deposit()` แล้วทดลองฝากเงินอีก 1,000 บาท ผลลัพธ์ใน Postman จะได้รับ Status 500 Internal Server Error และเมื่อตรวจสอบยอดเงินในบัญชีด้วย `GET /accounts/1` พบว่ายอดเงินยังคงเป็น 2000.0 เท่าเดิม รวมถึงตาราง `deposit_transaction` ใน pgAdmin ยังคงมีเพียง 1 แถวเท่าเดิม ไม่มีการเพิ่มแถวที่ 2 แสดงว่า `@Transactional` ได้สั่ง Rollback ยกเลิกการเปลี่ยนแปลงทั้งหมด ในฐานข้อมูลทันทีที่เกิดข้อผิดพลาด

## 6. เปรียบเทียบกรณีมี `@Transactional` กับไม่มี `@Transactional`

Overview **POST** http://localhost:8080/ No environment

**http://localhost:8080/accounts/1/deposit** Save Share

**POST** Send http://localhost:8080/accounts/1/deposit

Docs Params Authorization Headers 8 **Body** Scripts Settings Cookies

raw JSON Schema Beautify

```
1 {"amount": 1000}
```

**Body** Cookies Headers 4 Test Results 500 Internal Server Error • 226 ms • 249 B • Investigate server error Diagnose Decline

{} JSON Preview Debug with AI

```
1 {
2   "timestamp": "2026-09-15T17:54:20.483Z",
3   "status": 500,
4   "error": "Internal Server Error",
5   "path": "/accounts/1/deposit"
6 }
```

lab9/postgres@PostgreSQL 18

Query Query History Scratch Pad

```
1 SELECT * FROM deposit_transaction;
2
```

Data Output Messages Notifications

Showing rows: 1 to 3 of 1

|   | id<br>[PK] bigint | amount<br>double precision | account_id<br>bigint |
|---|-------------------|----------------------------|----------------------|
| 1 | 1                 | 1000                       | 1                    |
| 2 | 2                 | 1000                       | 1                    |
| 3 | 4                 | 1000                       | 1                    |

**อธิบายผลการทดลอง:**

- เมื่อมี @Transactional: คำสั่งทั้งหมดในเมธอดถูกนับเป็นธุรกรรมเดียวกันแบบ All or Nothing เมื่อเกิด Error ระบบจะ Rollback ยกเลิกข้อมูลทั้งหมด ทำให้ข้อมูลในฐานข้อมูลกลับไปสู่สถานะเดิม รักษาความถูกต้องสอดคล้องของข้อมูล (Data Consistency) ได้อย่างสมบูรณ์
- เมื่อไม่มี @Transactional: คำสั่ง save() ที่ทำงานไปก่อนหน้าจะถูกบันทึกลงฐานข้อมูลทันทีแบบแยกอิสระ เมื่อเกิด Exception ในตอนท้าย ข้อมูลที่บันทึกไปแล้วจะไม่ถูกย้อนกลับ ทำให้ข้อมูลค้างอยู่ครึ่งทาง เกิดความไม่ถูกต้องของข้อมูล (Inconsistency) เช่น เงินเพิ่มขึ้นแต่ไม่มีสลิปประวัติการฝาก

## 7. ตอบคำถามท้าย Lab ทั้ง 6 ข้อ

### 1. @Transactional มีหน้าที่อะไร?

**ตอบ:** ทำหน้าที่กำหนดขอบเขตของธุรกรรม (Transaction Boundary) ในระดับเมธอดแบบ Declarative โดยรวมคำสั่งการทำงานกับฐานข้อมูลหลายๆ คำสั่งเข้าด้วยกันเป็นก้อนเดียว (All or Nothing) หากทุกคำสั่งทำงานสำเร็จจะทำการ COMMIT บันทึกข้อมูลจริง แต่หากเกิดข้อผิดพลาด (RuntimeException) ระบบจะสั่ง ROLLBACK ยกเลิกและย้อนคืนข้อมูลทั้งหมดกลับสู่สถานะเดิมอัตโนมัติ

### 2. เพราะเหตุใดการฝากเงินจึงควรใช้ Transaction?

**ตอบ:** เพราะการฝากเงินประกอบด้วยขั้นตอนสำคัญ 2 ขั้นตอน คือ (1) การอัปเดตยอดเงินในบัญชี และ (2) การบันทึกประวัติการฝากเงิน ซึ่งทั้งสองขั้นตอนนี้ต้องสำเร็จร่วมกันเสมอ หากไม่ใช้ Transaction แล้วระบบขัดข้องระหว่างขั้นตอน จะทำให้เกิดข้อมูลขัดแย้ง เช่น ยอดเงินเข้าบัญชีแต่ไม่มีประวัติ หรือมีประวัติแต่ยอดเงินไม่เข้า การใช้ Transaction จึงช่วยการันตีความถูกต้องของข้อมูลทางการเงิน

### 3. COMMIT และ ROLLBACK ต่างกันอย่างไร?

**ตอบ:** • COMMIT: คือ การยืนยันและบันทึกการเปลี่ยนแปลงของข้อมูลทั้งหมดใน Transaction ลงฐานข้อมูลจริงอย่างถาวร เมื่อทุกขั้นตอนประมวลผลสำเร็จสมบูรณ์  
• ROLLBACK: คือ การยกเลิกและย้อนคืนการเปลี่ยนแปลงทั้งหมดที่เกิดขึ้นใน Transaction กลับไปเป็นสถานะเดิมก่อนเริ่มทำรายการ เสมือนว่าไม่เคยมีคำสั่งใดเกิดขึ้นเมื่อพบข้อผิดพลาด

### 4. Account และ DepositTransaction มีความสัมพันธ์แบบใด?

**ตอบ:** มีความสัมพันธ์แบบ One-to-Many (1 ต่อ หลาย) โดยบัญชีธนาคาร 1 บัญชี (Account) สามารถมีประวัติรายการฝากเงิน (DepositTransaction) ได้หลายรายการ ในระดับฐานข้อมูลจะจัดเก็บ Foreign Key (account\_id) ไว้ที่ตารางฝั่ง Many (deposit\_transaction)

### 5. เมื่อเกิด Error ขณะใช้ @Transactional ข้อมูลใน Database เปลี่ยนแปลงอย่างไร?

**ตอบ:** ข้อมูลใน Database จะไม่มีการเปลี่ยนแปลงใดๆ ทั้งสิ้น โดยระบบจะ Rollback ยกเลิกคำสั่งทั้งหมดที่ทำไปใน Transaction นั้น ทำให้ข้อมูลในทุกตารางกลับคืนสู่สถานะเดิมก่อนเริ่มทำรายการ

## 6. จากการทดลอง เมื่อลบ @Transactional ออก ผลลัพธ์แตกต่างจากเดิมอย่างไร?

**ตอบ:** เมื่อไม่มี @Transactional คำสั่ง save() แต่ละคำสั่งจะบันทึกลงฐานข้อมูลแยกกันทันที (Auto-commit) ดังนั้นเมื่อเกิด Error ในตอนท้าย ข้อมูลส่วนที่ถูกบันทึกไปก่อนหน้านี้จะไม่ถูกย้อนกลับ ส่งผลให้มีข้อมูลบางส่วนค้างอยู่ในฐานข้อมูล เกิดความไม่สอดคล้องกันของข้อมูล (Data Inconsistency)